

Input-Output (I/O) Systems: Full Lesson Breakdown

1. Why is Interfacing I/O Complex?

Connecting I/O devices (peripherals) is much harder than connecting memory because the outside world is chaotic compared to the CPU. The main reasons are:

- **Wide Variety:** Keyboards, mice, disks, and printers all operate differently.
- **Varying Speeds:** I/O devices are massively slower than the CPU and main memory.
- **Irregular Data Rates:** A user typing on a keyboard is unpredictable compared to a steady memory transfer fetch.
- **Varying Data Sizes:** A keyboard sends a few bytes; a hard disk sends entire kilobytes or megabytes at once.

2. The Solution: The I/O Module

Because of these differences, the CPU cannot talk directly to a peripheral. It uses a "middleman" called an **I/O Module** (or programmable I/O interface). The I/O module connects to the CPU/Memory via the System Bus on one side, and connects to the peripheral devices on the other side.

I/O Module Schematic

- **CPU Side (System Bus):** Connects to Data Lines, Address Lines, and Control Lines.
- **Inside the Module:** Contains Data Registers (to buffer data), Status/Control Registers (to know if the device is ready), and I/O Logic.
- **Device Side:** Connects to the peripheral via Data, Status, and Control signals.

3. Typical Steps During I/O Transfer

1. The Processor requests the I/O Module for the device's status.
2. The I/O Module checks the device and returns the status to the processor.
3. If the device is ready, the processor requests a data transfer.
4. The I/O Module gets the data from the external device.
5. The I/O Module transfers the data to the processor.
6. The Processor stores the data in main memory.

4. Input and Output Ports

I/O devices are interfaced through specific hardware gateways called ports:

- **Output Port:** Uses a PIPO (Parallel In, Parallel Out) register. It captures data from the data bus and holds it steady for the output device.
- **Input Port:** Uses a Tri-state Bus Driver. It only lets data flow from the input device onto the system bus when the specific input address is enabled.

5. Memory-Mapped vs. I/O-Mapped Interfaces

Feature	Memory-Mapped I/O	I/O-Mapped (Isolated) I/O
Address Space	Shares the same address space as Main Memory.	Has its own separate address space.
Instructions Used	Standard memory instructions (LOAD, STORE).	Special I/O instructions (IN, OUT).
Address Decoders	One shared decoder for both memory and I/O.	Separate address decoders for memory and I/O.
CPU Control Signals	No special signals needed.	CPU needs a special signal to specify if an address is for memory or I/O.

6. Direct Memory Access (DMA)

Programmed I/O (where the CPU handles every single byte) is terrible for fast, high-volume devices like Hard Disks. The CPU wastes too much time executing instructions just to move data.

The Fix: A hardware chip called a **DMA Controller**.

- The CPU tells the DMA Controller: *"Move this block of disk data to this memory address, and tell me when you are done."*
- The CPU goes to do other work.
- The DMA Controller takes over the system bus and moves the data directly into memory.
- When finished, the DMA sends an **Interrupt** to the CPU.

DMA Transfer Modes

- **Cycle Stealing:** The DMA takes the bus for just 1 or 2 cycles at a time (at "DMA breakpoints" between machine cycles) while the CPU isn't using the memory. The CPU continues working, just slightly slower.
- **Block Transfer:** The DMA takes full control of the bus and transfers the entire block of data at once. It is the fastest transfer rate, but the CPU sits completely idle waiting for the bus.